# Assignment 1: Project Specification and Analysis

## 1. Project Identification

| | |
|---|---|
| **Team Name** | Nazal |
| **Team Members** | Naman Anshumaan and Mohammad Fazlur Rahman |
| **Roll Numbers** | 1020241465 and 1020242018 |
| **Assigned Project** | Interactive Multi-Track Audio Editor and Mixer |
| **Repository Link** | https://github.com/Ashoka-CS-3810-Design-Practices-in-CS/project-1-nazal.git |

**Project Title:** Interactive Multi-Track Audio Editor and Mixer

**Problem Statement**

Editing and arranging audio typically requires specialized software that can be complex for users who have no experience with highly used professional digital audio editors. The project aims to develop an easy-to-use multi-track audio editor that allows users to import audio files, arrange them on a timeline, edit individual clips, mix multiple tracks, apply basic effects, and export the final composition. Users will be able to work on a project as a collection of multiple audio clips positioned on a timeline. They can directly manipulate these clips through an easily understandable UI while simultaneously controlling audio properties such as volume, mute, and solo.

**Motivation**

The project is meant to combine audio editing with interactive software design. The project enables the ability to visually comprehend audio editing, instead of treating audio processing as a sequence of command-line operations. The project also provides an opportunity to apply concepts from data structures, algorithms, concurrency, data representation, and modular software architecture to a practical application.

**Intended users**

Our intended users are the one who want to perform basic audio editing and mixing without requiring a high level audio workstation, which usually takes some time to understand. Examples of the users could be students working on projects, beginners wanting to learn basic audio editing, users creating simple music compositions, users editing voice recordings, podcasts, or other multi-track audio and users who need to combine and arrange multiple audio recordings.

**Main goal**

The main goal is to develop an intuitive, responsive, and modular application that allows users to create a complete multi-track audio composition by importing, arranging, editing, mixing, and exporting audio. The application should integrate the timeline, waveform visualization, audio playback, editing operations, effects, mixing controls, and export system into a single interface.

# 2. Requirements Analysis

## 2.1 User goals

Our application is intended primarily for beginner and intermediate users who need a basic and easy to understand platform for multi-track audio editing and mixing. A user should be able to:

1. Create and manage an audio project.
2. Import and arrange audio by adding clips to different tracks and positioning them on the timeline.
3. Edit audio clips with the help of basic actions such as trimming and splitting.
4. Mix multiple tracks by being able to control volume and other audio features.
5. Play and preview the composition while navigating through the timeline.
6. Apply basic audio effects, such as fade-in and fade-out.
7. Save, close the app and reopen projects while preserving the project settings.
8. Export the completed composition as an audio file.

The user will be able to provide information such as audio files, track names, clip positions on the timeline, editing selections, track volume levels, mute/solo settings, fade-in and fade-out parameters, project save locations and export settings. Based on this, the system should provide visual feedback. This would include imported audio clips on the timeline, waveform representations of the audio clip, current playback cursor, selected clips, track volume settings, mute/solo states, playback status, export progress and error messages. A successful interaction would be when the requested operation is completed correctly and the interface reflects the resulting state. For example, if a user moves an audio clip from 10 seconds to 20 seconds, the clip should visually move to the new position and subsequent playback should begin that clip at 20 seconds.

## 2.2 Functional Requirements

The system shall provide the following major functional requirements:

| **ID** | **Functional requirement** | **User input / action** | **System processing** | **Expected output** |
|---|---|---|---|---|
| FR1 | Project and track management | Create a project and create, rename, or delete tracks. | The system creates and maintains the project structure and track information. | A project containing the required tracks is displayed on the timeline. |
| FR2 | Audio import and arrangement | Import audio files and place clips on selected tracks. | The system validates the audio file, creates a clip representation, and assigns it a position on the timeline. | The audio clip appears on the selected track with its duration and waveform. |
| FR3 | Timeline editing | Select, move, trim, split, or remove portions of audio clips. | The system updates clip positions and boundaries while maintaining the project timeline. | The timeline reflects the edited arrangement and selected audio regions. |

| ID | Functional requirement | User input / action | System processing | Expected output |
|---|---|---|---|---|
| FR4 | Multi-track playback and navigation | Play, pause, stop, resume, or seek through the composition. | The system synchronizes all active tracks according to their timeline positions and controls playback from a common timeline. | All relevant tracks play together with a visible timeline cursor and playback feedback. |
| FR5 | Track mixing and effects | adjust track volume, mute or solo tracks, and apply fade-in/fade-out effects. | The system applies the selected mixing settings and effects during playback and rendering. | The user hears and sees the composition with the selected track settings and effects. |
| FR6 | Project saving and reopening | Save a project and reopen an existing project. | The system stores and restores the project structure, clip positions, and track settings. | The project reopens with its previous arrangement and settings preserved. |
| FR7 | Audio export | Export the completed composition and select available export settings. | The system combines the tracks, applies mixing and effects, and renders the final audio. | A playable audio file is produced and the user receives feedback on the export status. |

The system shall only accept supported and valid audio file formats and shall handle files that are corrupted, unsupported, or cannot be read. Operations such as placing, moving, trimming, or splitting clips shall be restricted to valid timeline positions and audio regions. Playback shall handle situations where an audio file cannot be loaded or played correctly and shall provide appropriate feedback to the user. Saving and reopening shall handle invalid or inaccessible project files without losing the current project state. During export, the system shall detect rendering or file-writing failures and inform the user if the final audio file cannot be generated. The system should also prevent invalid operations and provide clear feedback whenever an operation cannot be completed.

### 2.3 Error conditions and exception handling

| ID | Error condition | detection | Handling / feedback |
|---|---|---|---|
| ER1 | Unsupported audio format | File format/decoder validation fails | Import is rejected and the user is informed that the format is unsupported. |
| ER2 | Corrupted or invalid audio file | Audio decoder fails while reading the file | Import is cancelled and an error message is displayed. |
| ER3 | Audio playback failure | Playback subsystem reports an error | Playback is stopped safely and the user receives an error message rather than the application crashing. |
| ER4 | Invalid editing operation | Selected region or clip state does not satisfy operation requirements | Operation is cancelled and the user receives appropriate feedback. |

| ID | Error condition | detection | Handling / feedback |
|---|---|---|---|
| ER5 | Project file cannot be opened | Project parser or referenced audio files cannot be loaded | Project loading is cancelled or partially recovered where possible, with an explanation of the problem. |
| ER6 | Project save failure | File-system operation fails | User is informed that the project could not be saved and the current in-memory project remains unchanged. |
| ER7 | Export failure | Rendering/encoding process fails | Export is cancelled and the user receives an error message. |
| ER8 | Missing audio asset | Saved project references an unavailable audio file | System identifies the missing file and informs the user rather than crashing. |
| ER9 | Invalid effect parameters | User enters an invalid fade duration or value | Input is rejected or constrained to the valid range and the user receives visual feedback. |

# 3. Functional Specification

## 3.1 System Behaviour

The application is organized around five main workflows.

### A. Importing an Audio File

Select audio file → validate and load file → generate waveform → place clip on a selected track → display the clip on the timeline.

Once imported, the clip becomes part of the project and can be positioned or edited on the timeline.

### B. Editing a Clip

Select clip → select audio region → choose trim, split, or remove → update the clip → refresh the waveform and timeline.

The updated clip state is then used during playback.

### C. Multi-Track Playback

Press Play → read current timeline position → identify active clips → apply fade effects and track settings → mix active tracks → send audio to output → update playback cursor.

All tracks follow the same timeline position so that playback remains synchronized.

### D. Saving a Project

Select Save → store track information → store clip positions and durations → store volume, mute, solo, and fade settings → store audio references → save project.

This allows the same arrangement and settings to be restored when the project is reopened.

### E. Exporting

Select Export → process clips according to the timeline → apply edits, fades, and track settings → mix the tracks → encode the final audio → save output file → show export feedback.

## 3.2 Use Cases

The following use cases represent the main interactions supported by the application.

| ID | Use Case | Actor | Steps | Expected result |
|---|---|---|---|---|
| UC1 | Edit audio clip | User | Select clip → Select region → Trim/split/remove | Timeline and waveform reflect the editing operation and playback uses the modified region. |
| UC2 | Mix multiple tracks | User | Create tracks → Add clips → Adjust volume → Mute/solo tracks → Play | Multiple tracks play synchronously according to their timeline positions and mixing settings. |
| UC3 | Apply fade effect | User | Select clip → Choose fade-in/fade-out → Configure duration | Clip playback gradually increases/decreases in volume over the specified region. |

| ID | Use Case | Actor | Steps | Expected result |
|---|---|---|---|---|
| UC4 | Export composition | User | Complete composition → Select export → Choose format/settings → Export | System renders and saves the final mixed composition and provides feedback. |

### 3.3 Non-Functional Requirements

**Performance and responsiveness**

The interface should remain responsive during playback and normal timeline interaction. Actions such as moving or selecting clips should respond without noticeable delay, while waveform generation and audio processing should not unnecessarily block the interface. Longer operations such as exporting should provide progress or status feedback, and multiple tracks should remain synchronized during playback.

**Correctness and reliability**

Audio clips should play at their correct timeline positions, and track volume, mute, and solo settings should affect both playback and export consistently. Editing operations should correctly update the project state as well as the visual timeline. Invalid files and playback or export failures should be handled without causing the application to crash.

**Usability and accessibility**

The timeline, track controls, playback controls, and effects should be clearly separated and easy to understand. Users should be able to manipulate clips directly on the timeline, while selected clips, audio regions, and the current playback position should have clear visual feedback. Controls should also use simple labels and predictable behaviour.

**Scalability**

The project structure should support multiple tracks and multiple clips per track without requiring major changes to the core design. The architecture should also allow additional effects, audio formats, and editing operations to be added later.

**Maintainability**

The application should keep audio data, project management, tracks and clips, timeline handling, playback, waveform processing, effects, user interface, and export as separate parts of the system. Keeping these responsibilities separate will make the code easier to understand, modify, and maintain.

**Modularity**

Audio effects should remain separate from playback and project-management logic so that new effects can be added without major changes to other components. Similarly, audio importing and decoding should remain separate from the timeline and user interface.

**Portability**

The application should use audio-processing and user-interface technologies that work on the platform selected by the team. The exact platform and framework will depend on the implementation technology chosen for the project.

**Security and resource handling**

Invalid files should be handled safely without crashing the application. Audio resources should be released when they are no longer required, and large audio files should be handled carefully to avoid unnecessary memory usage. Temporary files created during processing or export should be cleaned up, and file paths used while loading and saving projects should be handled safely.

# 4. Project-Specific Technical Analysis

## 4.1 Track and audio clip representation

The project will be represented as a collection of tracks, with each track containing its own audio clips and track-level settings.

Each clip will store information such as its audio source, start position on the timeline, duration, and the portion of the source audio being used. Each track will store its volume, mute, and solo state.

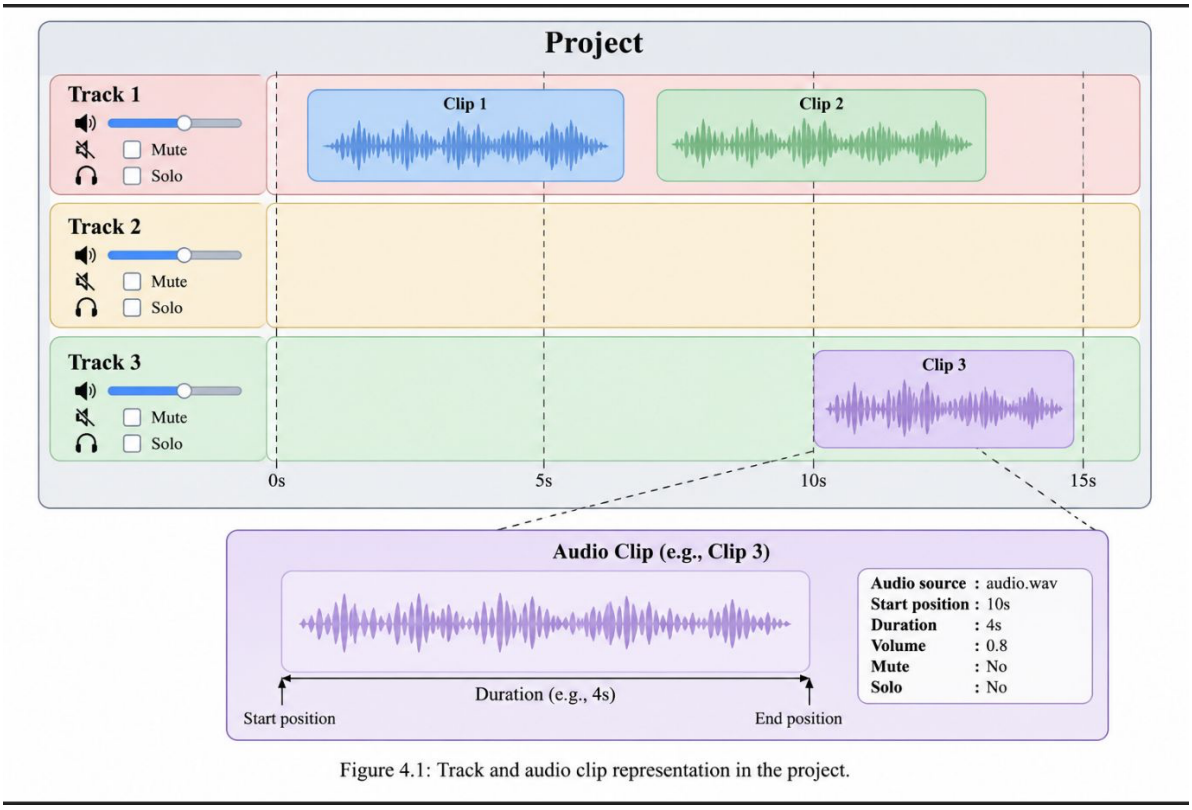



Figure 4.1: Track and audio clip representation in the project.

Figure 4.1: Track and audio clip representation in the project.[1]

## 4.2 Multi-track timeline

The timeline will use a common time coordinate system shared by all tracks. Each clip will have a start position and duration, allowing clips to be moved and arranged independently while remaining synchronized with the global timeline. When a clip is moved, only its timeline position needs to be updated; its underlying audio data does not need to be modified.

## 4.3 Computation of intervals and free gaps

Playback will be controlled using a single global playback position. The playback engine will determine which clips are active at the current position and read the corresponding audio data from each track.

[1]This figure was generated using AI.

The active audio streams will then be mixed together before being sent to the audio output. This ensures that clips on different tracks remain synchronized.

### 4.4 Waveform generation

When an audio clip is imported, its waveform will be generated from the audio samples and displayed alongside the clip on the timeline. For efficient display, samples can be grouped into small timeline regions and represented using their minimum and maximum amplitudes. The waveform is therefore a visual representation of the same audio data used by the playback engine.

### 4.5 Trimming, splitting and removing audio

Editing operations will primarily modify the clip's representation rather than permanently modifying the original audio file.

- **Trim:** changes the start or end boundaries of a clip.
- **Split:** creates two clip representations from different portions of the same source audio.
- **Remove:** removes a clip or selected region from the project.

### 4.6 Volume, mute and solo

Each track will maintain its own volume level, mute state, and solo state. During playback, the audio engine will apply these settings before mixing the tracks. If a track is muted, its audio will not contribute to the output. If one or more tracks are soloed, only the appropriate soloed tracks will be included in the mix.

### 4.7 Fade-in and fade-out

Fade effects will be represented as properties of a clip, including the fade type and duration. During playback or export, the audio engine will apply a gradually changing gain to the affected portion of the clip.

### 4.8 Saving and reopening projects

The project file will store the project structure rather than the processed final audio. It will preserve track information, clip positions and durations, source audio references, volume settings, mute/solo states, and fade settings. When a project is reopened, these values will be loaded and used to reconstruct the timeline and playback state.

### 4.9 Rendering and export

For export, the system will process the project from the beginning of the timeline. The audio from all active tracks will be combined according to their positions, volume settings, mute/solo states, and effects.

### 4.10 Modular architecture

The implementation will be divided into modules so that individual components can be developed and tested independently.

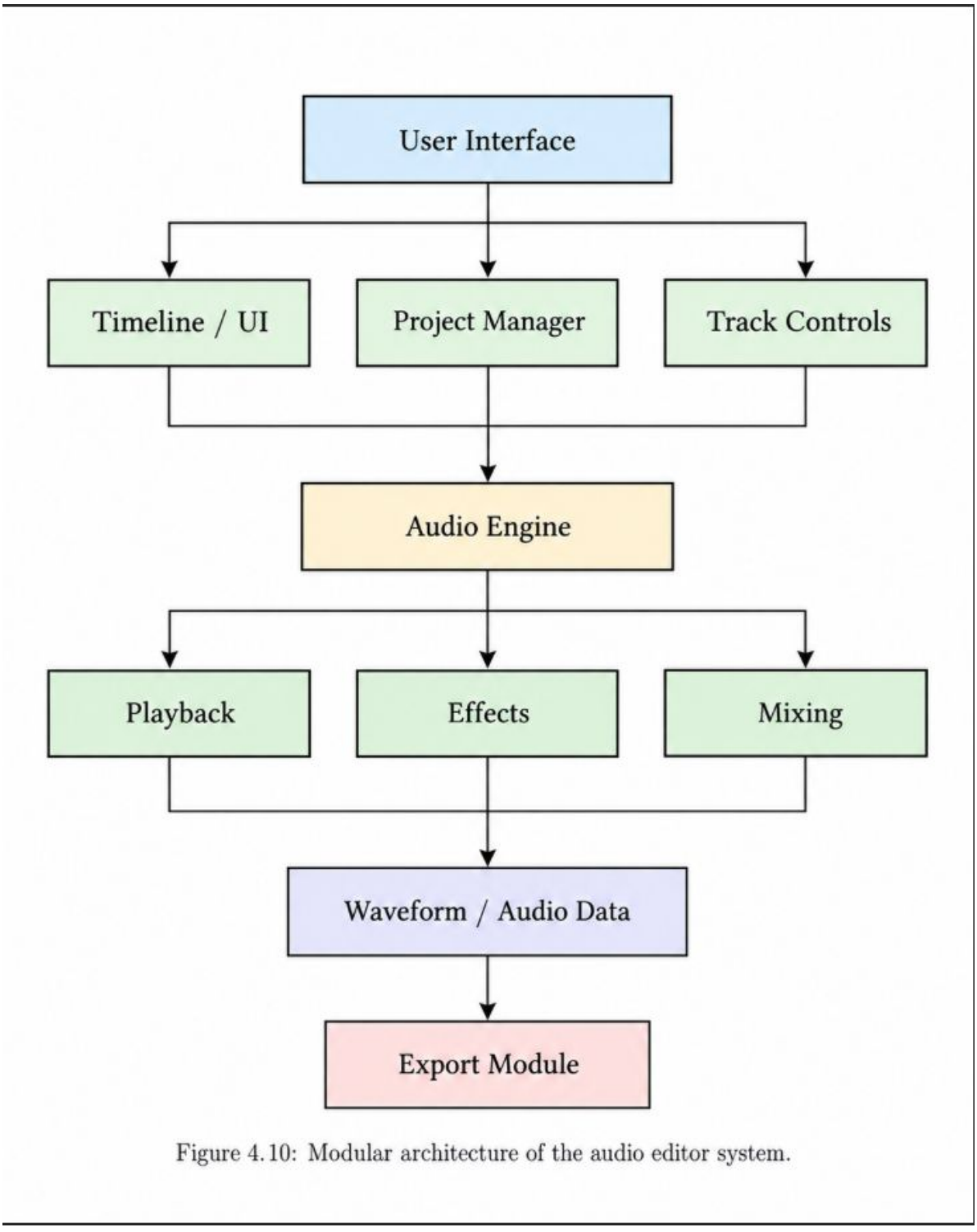



Figure 4.10: Modular architecture of the audio editor system.

Figure 4.10: Modular architecture of the application.[2]

The UI handles user interaction and visual feedback, the project manager handles project and timeline data, the audio Engine handles playback and mixing, the Waveform module handles visual audio representation, and the export module generates the final composition. This separation

[2]This figure was generated using AI.

improves maintainability and allows changes to one component without requiring major changes to the others.

# 5. Computational Theory

### Data representation and abstraction

Data abstraction involves representing a complex real-world object using a structured representation that contains the information necessary for the application. The audio editor represents projects, tracks, audio clips, timeline positions, audio settings and effects as structured data objects. For example, instead of treating every audio sample as an independent object in the UI, the application represents an audio clip using metadata such as its timeline position, source region, duration, and effects.

It is appropriate because this abstraction separates what the audio project represents from how it is displayed or processed. It allows the same project representation to be used by timeline UI, playback engine, project saving and export system.

The design assumes that the chosen representation can efficiently reference the required audio data. Very large audio files may require careful memory management.

### Interval processing

An interval represents a range between a starting point and an ending point. An audio clip can be represented as: [start time, end time]. If a clip starts at 10 seconds and lasts 5 seconds: [10, 15], the playback engine can determine whether a clip is active at time $t$ by checking whether:

$$\text{start} \leq t < \text{end}$$

Intervals are also useful for detecting overlapping clips, determining which clips should play, splitting clips, finding selected regions and processing timeline edits. It is appropriate because audio clips naturally occupy continuous regions of the timeline. Representing them as intervals gives the system a simple way to reason about their positions and overlaps.

A limitation could be that this representation primarily describes temporal placement. It does not itself represent the actual audio samples, which must be stored separately.

### Concurrency and asynchronous execution

Concurrency allows multiple activities to make progress during overlapping periods of execution. The application may need to perform tasks such as audio playback, UI updates, waveform generation, audio processing, file loading and exporting without allowing expensive processing to freeze the interface.

For example:

```
Main/UI thread
  |
  +-- User interaction
  +-- Timeline updates

Background/audio processing
  |
  +-- Waveform generation
  +-- File processing
  +-- Export rendering
```

```
Audio subsystem
  |
  +-- Playback
```

It is appropriate because audio playback requires timely processing, while operations such as waveform generation and exporting can take significant time. Separating these operations helps maintain a responsive UI. However, a current limitation is that concurrency introduces synchronization challenges. Shared project data must be accessed safely so that editing the project while playback or processing is occurring does not create inconsistent state.

**Complexity analysis**

Complexity analysis describes how the computational cost of an operation changes as the amount of input data increases. Complexity is relevant to operations such as:

- Finding clips active at a timeline position.
- Rendering many clips.
- Generating waveform representations.
- Loading large audio files.
- Searching tracks/clips.

For example, if every playback update scans every clip in every track, the work grows with the total number of clips. If there are $N$ clips, a straightforward implementation may require approximately $O(N)$ work to determine active clips at a particular time.

It is appropriate because a multi-track project can contain many clips. Understanding computational cost helps us design an implementation that remains responsive as projects become larger. However, it should be noted that the exact complexity depends on the data structures used. More sophisticated structures could reduce the amount of work required to locate active clips, but would increase implementation complexity.